Guidewire InsuranceSuite™ for Guidewire Cloud

Workflow Configuration

Release: 2025.11.0

GUIDEWIRE

Product Name: Guidewire InsuranceSuite for Guidewire Cloud
Product Release: 2025.11.0
Document Name: Workflow Configuration
Document Revision: 05-December-2025

Contents

Support **5**

Part 1
Workflow configuration **7**

1 **Using the workflow editor** 9
Workflow in Guidewire Studio 10
Access the workflow editor 10
Workflow editor window 10
Workflow editor elements 12
Understanding workflow steps 13
Using the workflow right-click menu 13
Using search with workflow 14
2 **Guidewire workflow** 15
Understanding workflow 15
Workflow instances 15
Work items 17
Workflow process format 17
Workflow step summary 17
Workflow Gosu 18
Workflow versioning 18
Workflow localization 20
Workflow structural elements 20
<Context> workflow element 20
<Start> workflow element 21
<Finish> workflow element 21
Common step elements 21
Enter and exit scripts in workflows 21
Asserts in workflows 22
Events in workflows 22
Notifications in workflows 22
Branch IDs in workflows 23
Basic workflow steps 23
AutoStep workflow step 23
MessageStep workflow step 24
ActivityStep workflow step 25
ManualStep workflow step 26
Outcome workflow step 27
Step branches 28
Working with branch IDs 29
GO 29
TRIGGER 30
TIMEOUT 31
Creating new workflows 33
Clone an existing workflow 33

Extend an existing workflow..... 33
Extending a workflow: a simple example..... 34
Instantiating a workflow..... 37
A simple example of instantiation..... 37
The workflow engine..... 39
Distributed execution..... 39
Synchronicity, transactions, and errors..... 40
Workflow subflows..... 43
Workflow administration..... 43
Workflow debugging, logging, and testing..... 45
Process logging..... 45
Workflow testing in PolicyCenter and BillingCenter..... 45
Enable the ITestingClock plugin in PolicyCenter and BillingCenter..... 45

Support

For assistance, visit the Guidewire Community.

Guidewire customers

https://community.guidewire.com

Guidewire partners

https://partner.guidewire.com

part 1

Workflow configuration

You can configure workflow in the application.

chapter 1

Using the workflow editor

InsuranceSuite applications use workflow in different ways.

Workflow in ClaimCenter

Guidewire ClaimCenter uses workflow primarily to support `MetroReport` integration. Guidewire defines and stores each base configuration workflow process as a separate file in the following directory:

```
ClaimCenter/modules/cc/config/workflow
```

Each file name corresponds to the workflow process that it defines (for example, `MetroReport.1.xml`). Each workflow file name contains a version number. If you create a new workflow, Studio creates a workflow file with version number 1. If you modify an existing base configuration workflow, Studio creates a copy of the file and increments the version number. In each case, Studio places the workflow file in the following directory:

```
ClaimCenter/modules/configuration/config/workflow
```

Workflow in PolicyCenter

Guidewire PolicyCenter uses workflow to drive various key business processes (Renewals, Policy Changes, Submissions, and similar items). Guidewire defines and stores each base configuration workflow process as a separate file in the following directory:

```
PolicyCenter/modules/configuration/config/workflow
```

Each file name corresponds to the workflow process that it defines (for example, `CompleteCancellationWF.1.xml`). Each workflow file name contains a version number. If you create a new workflow, Studio creates a workflow file with version number 1. If you modify an existing base configuration workflow, Studio creates a copy of the file and increments the version number. In each case, Studio places the workflow file in the following directory:

```
PolicyCenter/modules/configuration/config/workflow
```

Workflow in BillingCenter

Guidewire BillingCenter uses workflow to drive some of its payment processes. Guidewire defines and stores each base configuration workflow process as a separate file in the following directory:

```
BillingCenter/modules/bc/config/workflow
```

Each file name corresponds to the workflow process that it defines (for example, `CancelImmediately.1.xml`). Each workflow file name contains a version number. If you create a new workflow, Studio creates a workflow file with

version number 1. If you modify an existing base configuration workflow, Studio creates a copy of the file and increments the version number. In each case, Studio places the workflow file in the following directory:

```
BillingCenter/modules/configuration/config/workflow
```

Workflow in Guidewire Studio

Even though Guidewire defines the workflow scripts in XML files, you use Guidewire Studio to view, edit, manage, and create new workflows scripts. Thus, you do not work directly with XML files. Instead, you work with their representation in Guidewire Studio, in the Studio **Workflows** editor.

Access the workflow editor

Procedure

In Guidewire Studio, navigate to **configuration** > **config** > **Workflows**, and then select a workflow.

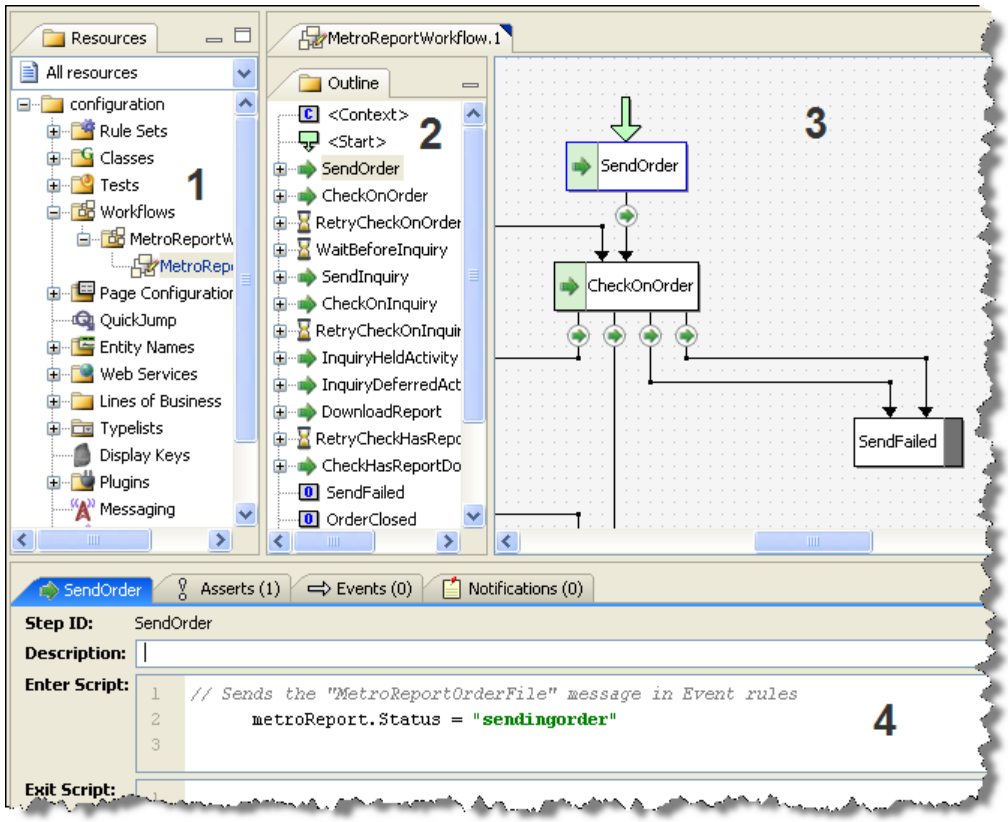
Workflow editor window

Within the **Workflows** editor, there are multiple work areas, each of which performs a specialized function:

Area	View	Description
1	Tree view	Studio displays each workflow type as a node in the **Resources** tree. If you have multiple versions of a workflow type, Studio displays each one with an incremental version number at the end of the file name.
2	Outline view	Studio displays an outline of the selected workflow process in the **Outline** pane. This outline lists all the steps and branches for the workflow in the order that they actually appear in the workflow XML file. You can re-order these steps as desired. You can also re-order the branches within a step. First, select an item, then right-click and select the appropriate menu item.
3	Layout view	Studio displays a graphical representation of the workflow in the workflow pane. You use this representation to visualize the workflow. You also use it to edit the defining values for each step and branch.
4	Property view	Studio displays detailed properties for the selected step or branch, much of which you can modify.

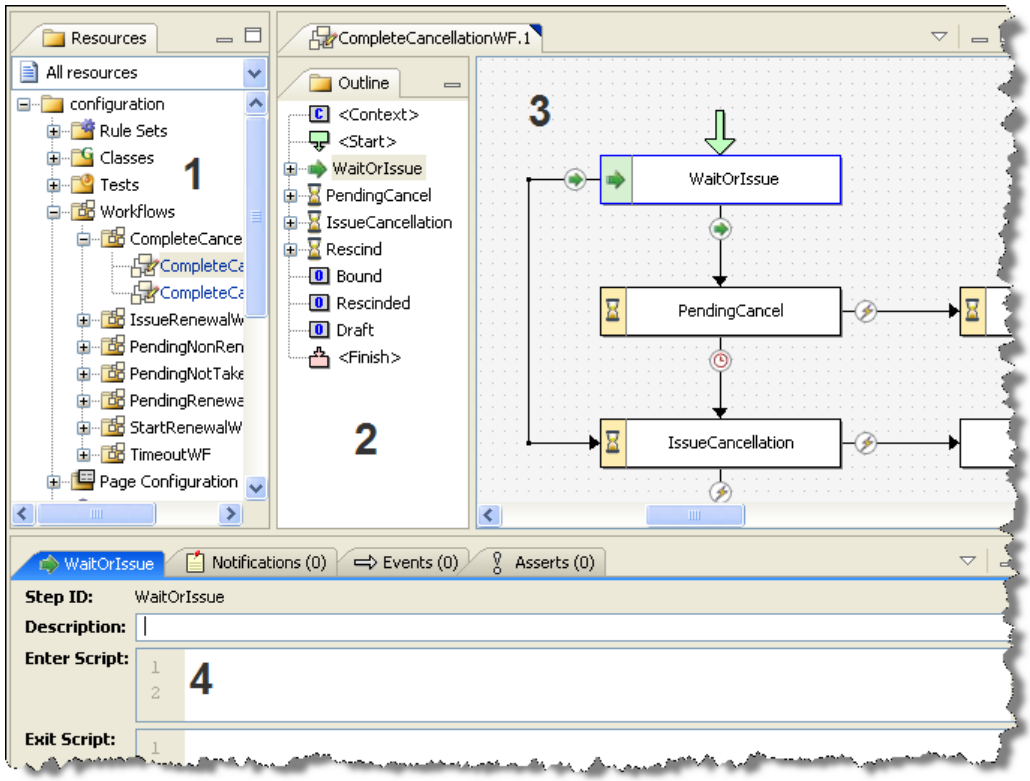
ClaimCenter workflow editor

For example, the ClaimCenter base configuration defines a `MetroReportWorkflow` script. In Studio, it looks similar to the following:



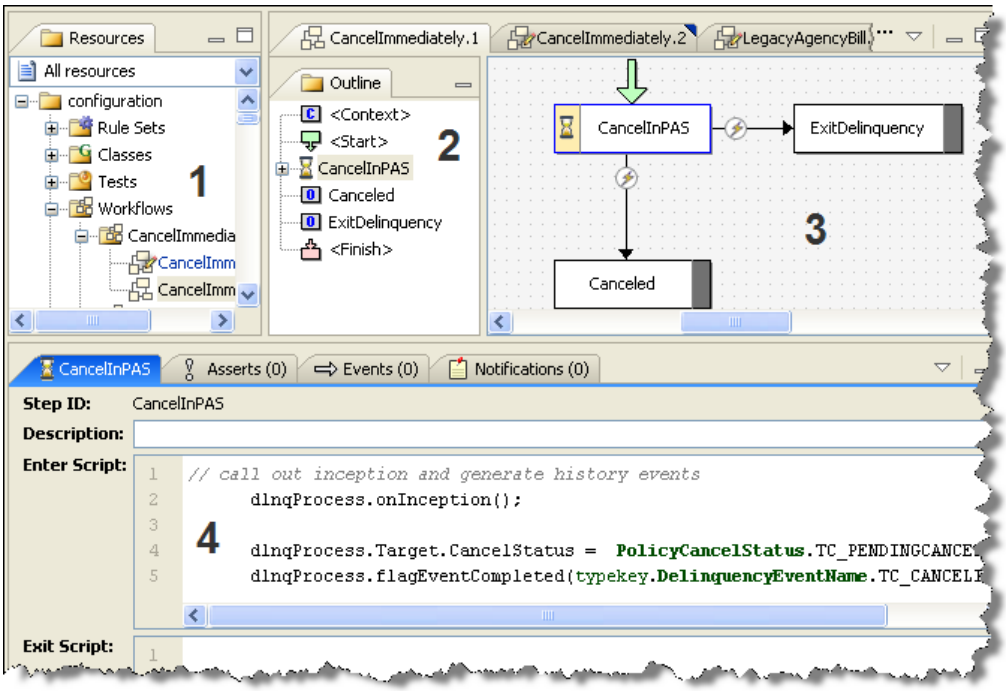
PolicyCenter workflow editor

For example, the PolicyCenter base configuration defines a `CompleteCancellationWF` script. In Studio, it looks similar to the following:



BillingCenter workflow editor

For example, the BillingCenter base configuration defines a `CancelImmediately` script. In Studio, it looks similar to the following:



Workflow editor elements

The following table lists the main workflow elements and describes each one.

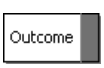
Element	Editor	Description	More information
`<Context>`		Every workflow begins with a `<Context>` block. You use it to conveniently define symbols that apply to the workflow.	"<Context> workflow element" on page 20
`<Start>`		Defines the step on which the workflow starts. It optionally contains Gosu blocks to set up the workflow or its business data. It runs before any other workflow step.	"<Start> workflow element" on page 21
`AutoStep`		Defines a workflow step that finishes immediately, without waiting for time to pass or for an external trigger to activate it.	"AutoStep workflow step" on page 23
`MessageStep`		Supports messaging-based integrations. It automatically generates and sends a single integration message and then stops the workflow until the message completes. (Typically, this is through receipt of an `ack` return message.) After the message completes, the workflow resumes automatically.	"MessageStep workflow step" on page 24
`ActivityStep`		An `ActivityStep` is similar an `AutoStep`, except that it can use any of the branch types, such as a `TRIGGER` or a `TIMEOUT`, to move to the next step. However, before an `ActivityStep` branches to the next step, it waits for one or more activities to complete.	"ActivityStep workflow step" on page 25
`ManualStep`		Defines a workflow step that waits for someone—or some thing—to invoke an external trigger or for some period of time to pass.	"ManualStep workflow step" on page 26
GO		Indicates a branch or transition to another workflow step. It occurs only within an `AutoStep` workflow step.	"GO" on page 29

Element	Editor	Description	More information
		• If there is only a single GO element within the workflow step, branching occurs immediately upon workflow reaching that point. • If there are multiple GO elements within the workflow step, each GO element (except the last one) must contain conditional logic. The workflow then determines the appropriate next step based on the defined conditions.	
`TRIGGER`		Indicates a branch or transition to another workflow element. It occurs only within a `ManualStep` workflow step. Branching occurs only upon manual invocation from outside the workflow.	"TRIGGER" on page 30
`TIMEOUT`		Indicates a branch or transition to another workflow element. It occurs only within a `ManualStep` workflow step. Branching to another workflow step occurs only after a specific time interval has passed.	"TIMEOUT" on page 31
`Outcome`		Indicates a possible outcome for the workflow. This step is special. It indicates that it is a last step, out of which no branch leaves.	"Outcome workflow step" on page 27
`<Finish>`		(Optional) Defines a Gosu script to run at the completion of the workflow to perform any last clean up after the workflow reaches an outcome. It runs after all other workflow steps.	"<Finish> workflow element" on page 21

Understanding workflow steps

Each workflow step represents a location in the workflow. It does not have a business meaning outside of the workflow. Therefore, it is permissible to use whatever IDs you want and arrange them however it is most convenient for you. (Beware of infinite cycles between steps. The application treats too many repetitions between steps as an error.)

A workflow script can contain any of the following steps. It must contains at least one `Outcome` step. It must also start with one each of the `<Context>` and `<Start>` steps described in "Workflow structural elements" on page 20.

Type	Workflow contains	Icon	Step	Description
`AutoStep`	Zero, one, or more			Step that the application guarantees to finish immediately. See "AutoStep workflow step" on page 23.
`ManualStep`	Zero, one, or more			Step that waits for an external `TRIGGER` to occur or a `TIMEOUT` to pass. See "ManualStep workflow step" on page 26.
`ActivityStep`	Zero, one, or more			Step that waits for one or more activities to complete before continuing. See "ActivityStep workflow step" on page 25.
`MessageStep`	Zero, one, or more			Special-purpose step designed to support messaging-based integrations. See "MessageStep workflow step" on page 24.
`Outcome`	One or more			Special final step that has no branches leading out of it. See "Outcome workflow step" on page 27.

Using the workflow right-click menu

You can modify a workflow step by first selecting it, then selecting different items from the right-click menu.

Desired result	Actions
To change a workflow step name	Select **Rename** from the right-click menu. This opens the **Rename StepID** dialog in which you can enter the new step name.
To change a workflow step type	Select **Change Step Type** from the right-click menu, then the type of workflow step from the submenu. This action opens a dialog in which you set the new workflow step type parameters.
To move a workflow step up or down	Select **Move Up** (**Move Down**) from the right-click menu. The editor only presents valid choices for you to select. This action moves the workflow step up or down within the workflow outline view.
To create a new branch	Select **New <BranchType>** from the right click menu. The editor presents you with valid branch types for the workflow step type. This action opens a dialog in which you set the new branch parameters.
To delete a workflow step	Select **Delete** from the right-click menu. This action removes the workflow step from the workflow outline. The workflow editor does not permit you to remove the workflow step that you designate as the workflow start step.

See also

- *Globalization Guide*

Using search with workflow

It is possible to search for a specific text string within a workflow by selecting **Find in Path** from the Studio **Edit** menu. You can search on a localized text strings as well. You can also select a workflow and select **Find in Path** from the right-click menu.

- If you use the **Search** menu option, you can filter the resources to check.
- If you use the right-click menu option, then the search encompasses all active resources.

In either case, Guidewire Studio opens a search pane at the bottom of the screen and displays any matches that it finds. You can click on a match to open the workflow in which the match exists.

chapter 2

Guidewire workflow

This topic covers application workflow. Workflow is the Guidewire generic component for executing custom business processes asynchronously.

Note: The Workflow component is not the same thing as the Autopilot Workflow Service in Guidewire Cloud. For information, see *Autopilot Workflow Service* documentation: https://docs.guidewire.com/cloudProducts/autopilotworkflowservice.

Understanding workflow

There are multiple aspects of workflow:

Term	Definition
workflow, workflow instance	A specific running instance of a particular business process. Guidewire persists a workflow instance to the database as an entity called `Workflow`.
workflow type	A single kind of flow process, such as a Cancellation workflow.
workflow process	A definition of a workflow type in XML. Guidewire defines workflow processes in XML files that you manage in Guidewire Studio through the graphical **Workflows** editor.

Discussions about *workflow* in general or the *workflow system* refer usually to the workflow infrastructure as a whole.

Workflow instances

The application saves a *workflow instance* as a row in the database marking the existence of a single running business flow. The application creates a workflow instance in response to a specific need to perform a task or function, usually asynchronously. The following are examples in the base configuration:

- ClaimCenter provides a ready-to-use integration to the Metropolitan Reporting Bureau (http://www.metroreporting.com) that it bases on workflow. You use this workflow as an aid in obtaining police reports of accidents.
- PolicyCenter creates a *macro* workflow as a companion to each newly-created Job (Submission, PolicyChange, Renewal, for example). This workflow is responsible for pushing the Job through its life cycle.
- BillingCenter uses workflows to manage agency billing and account and policy delinquencies.

The newly created instance takes the form of a database entity called `Workflow`. Because the application creates the `Workflow` entity in a bundle with other changes to its associated business data, the application does nothing with the

workflow until it commits the workflow. The application does not send messages to any external application unless the surrounding bundle commits successfully.

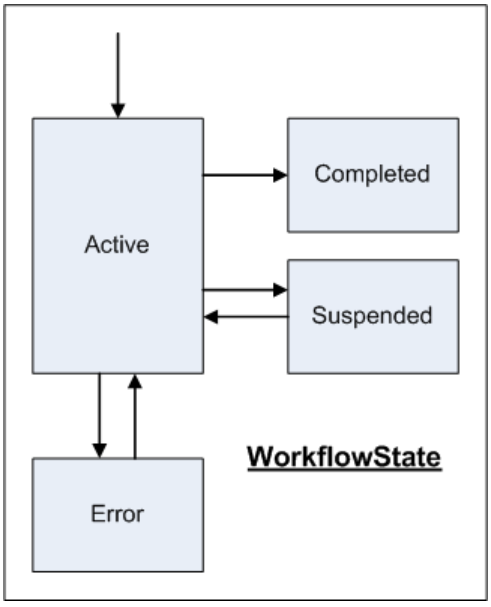
Note: For more information on the `Workflow` entity, consult the *Data Dictionary*.

After creation of the `Workflow` entity, nothing further happens from the viewpoint of the code that created the workflow. The workflow merely continues to execute asynchronously, in the background, until it completes. It is not possible, in code, to wait on the workflow, as you can wait for a code thread to complete, for example. This is because some workflows can literally and deliberately take months to complete.

All workflows have a *state* field ,a typekey of type `WorkflowState`, that tracks how the workflow is doing. This state, and the transitions between states, is simple:

- All newly beginning `Workflow` entities start in the `Active` state, meaning they are still running.
- If a `Workflow` entity finishes normally, it moves to the `Completed` state, which is final. A workflow in the `Completed` state takes no further action and exists from then on only as a record in the database.
- If you suspend a workflow, either from the application **Administration** interface, from the command line, or through the Workflow API, the workflow moves to the `Suspended` state. A workflow in the `Suspended` state does nothing until manually resumed from the **Administration** interface, from the command line, or through the Workflow API.
- If an error occurs to a workflow executing in the background, the workflow moves into the `Error` state after it attempts the specified number of retries. A workflow in the `Error` state does nothing until manually resumed from the **Administration** interface, the command line, or the Workflow API.

The following graphic illustrates the possible workflow states:



This diagram does not convey any information about how an active workflow, a workflow in the `Active` state, is actually processing. For active workflows, Guidewire defines the workflow state in the `WorkflowActiveState` typelist, which contains the following states:

- `Running`
- `WaitManual`
- `WaitActivity`
- `WaitMessage`

Whether the workflow is actually running depends on whether it is the current work item being processed.

Work items

Each running workflow instance can have a *work item*. If a running workflow does not have a work item associated with it, the workflow writer picks up the workflow instance at the next scheduled run. The state of this work item is one of the following:

- `Available`
- `Failed` – The application retries a `Failed` work item up to the maximum retry limit.
- `Checkedout` – The application processes a `Checkedout` work item in a specific worker's queue after the work item reaches the head of that queue.

Workflow process format

To structure a workflow script, Guidewire uses the concept of a directed graph that shows how the `Workflow` instance moves through the various states. This directed graph is known formally as a Petri net or P/T net. Guidewire calls each state a *Step* and calls a transition between two states a *Branch*. Guidewire defines multiple types of steps and branches.

Even though Guidewire defines the workflow scripts in XML files, you use Guidewire Studio to view, edit, manage, and create new workflows scripts.

See also

- "Workflow in Guidewire Studio" on page 10

Workflow step summary

The workflow process consists of the following steps (or states). The table lists the steps in the approximate order in which they occur in the workflow script. A designation of *structural* indicates that these steps are mandatory and that Studio inserts them into the workflow process automatically. Studio marks the structural steps with brackets (`<...>`) to indicate that they are actually XML elements. Some of the structural elements have no visual representation within the workflow diagram itself. You can choose them only from the workflow outline.

Step	Script contains	Description
"<Context> Workflow Element"	Exactly one	**Structural.** Element for defining symbols used in the workflow. Generally, you define a symbol to use as a convenience in defining objects in the workflow path. For example, you can define a symbol such that inserting `claim` into the workflow text actually inserts `Workflow.Claim`, or inserting `Cancellation` actually inserts `Workflow.PolicyPeriod.Cancellation`, or inserting `dlnqProcess` into the workflow text actually inserts `Workflow.DelinquencyProcess`.
"<Start> Workflow Element"	Exactly one	**Structural.** Element defining on which step the Workflow element starts. It can optionally contain Gosu code to set up the workflow or its business data.
"AutoStep Workflow Step" "ActivityStep Workflow Step" "ManualStep Workflow Step" "MessageStep Workflow Step"	Zero, one, or more	A step is one stage that the Workflow instance can be in at a time. There can be zero, one, or more of any of these steps, in any order. Each of these steps in turn can contain one or more of the following: • Any number of `Assert` code blocks for ensuring the conditions in the step are met. • An `Enter` block with Gosu code to execute on entering the step. • Any number of `Event` objects that generate on entering the step. • Any number of `Notification` objects that generate on entering the step. • An Exit block with Gosu code to execute on leaving a step. Several of these steps can contain other, step-specific, components: • An `ActivityStep` can contain any number of `Activity` steps that generate on entering the step.

Step	Script contains	Description
		• An `AutoStep` or `ActivityStep` can contain any number of `GO` branches which lead from this step to another step. • A `ManualStep` can contain any number of `TRIGGER` branches which lead from this step to another step, if something or someone from outside the workflow system manually invokes it. (This happens typically through the application interface.) • A `ManualStep` can contain any number of `TIMEOUT` branches that lead to another step after the elapse of a certain time.
"Outcome Workflow Step"	One or more	A specialized step that indicates a last step out of which no branch leaves.
"<Finish> Workflow Element"	Zero or one	**Structural.** An optional code block that contains Gosu code to perform any last cleanup after the workflow reaches an Outcome.

See also

- "Using the workflow editor" on page 9

Workflow Gosu

Workflow elements `Start`, `Finish`, `Enter`, `Exit`, `GO`, `TRIGGER`, and `TIMEOUT` can all contain embedded Gosu. The Workflow engine executes this Gosu code any time that it executes that element. The specific order of execution is:

- The Workflow engine runs `Start` before everything else
- The Workflow engine runs `Enter` on entering a step.
- The Workflow engine runs `Exit` upon leaving a step. It runs `Exit` before the branch leading to the next step. Thus, the actual execution logic from Step A to Step B is to Exit A, then do the Branch, then Enter B.
- The Workflow engine runs `GO`, `TRIGGER`, `TIMEOUT` elements as it encounters them upon following a branch.
- The Workflow engine runs `Finish` after it runs everything else.

Within the Gosu block, you can access the currently-executing workflow instance as `Workflow`. If you need to use local variables, declare them with `var` as usual in Gosu. However, if you need a value that persists from one step to another, create it as an extension field on `Workflow` and set its value from scripting. You can also create subflows in the Gosu blocks.

The current bundle for workflow actions is the bundle that the application uses to load the `Workflow` entity instance. The expression `Workflow.Bundle` returns the workflow bundle.

See also

- *Gosu Reference Guide*

Workflow versioning

After you create a workflow script and make it active, it can create hundreds or even thousands of working instances in the application application. As such, you do not want to modify the script as actual existing workflow instances can possibly be running against it. (This is similar to modifying a program while executing it. It can lead to very unpredictable results.)

However, you might choose to modify a script. Then, you would want all newly created instances of the workflow to use your new version of the script.

Guidewire stores each workflow script in a separate XML file. By convention, Guidewire names each file a variant of *xxx*`WF.#.xml`:

- *xxx* the workflow name (which is mixed cased `LikeThis`)

- `#` is the version number of the workflow process (starting from 1)

Every newly created (copied) workflow script has a different version number from its predecessor. (The higher the version number, the more recent the script.) Thus, a script file name of `ManualExecutionWF.2.xml` means workflow type `ManualExecution`, version 2. As the application creates new instances of the workflow script, it uses the most recent script—the highest-numbered one—to run the workflow instance against.

It is possible to start a specific workflow with a specific version number. For details, see "Instantiating a workflow" on page 37.

The Workflow engine enforces the following rules in regards to version numbers:

- If you create a new workflow instance for a given workflow subtype, thereafter, the Workflow engine uses the script with the highest version number. The application saves this number on the workflow instance as the `ProcessVersion` field.
- From then on, any time that the Workflow instance wakes up to execute, the Workflow engine uses the script with the same typecode and version number of the instance only.
- It is forbidden to have two workflow scripts with the same subtype and version number. The server refuses to start if you try.
- If a workflow instance cannot find a script with the right subtype and version number, it fails with an error and drops immediately into the `Error` state. (This might happen, perhaps, if someone inadvertently deleted the file or the file did not load for some reason.)

When to create a new workflow version

Guidewire recommends, as a general rule, that you create a new workflow version under most circumstances if you modify a workflow. For example:

- If you add a new step to the workflow, then create a new workflow version.
- If you remove an existing step from the workflow, then create a new workflow version.
- If you change the step type, such as from Manual to an automatic step type, then create a new workflow version.

More specifically, for each workflow, the application does the following:

- Records the current step of an active workflow in the database. Each change to the basic structure of a workflow requires a new version.
- Records the branch that an active workflow selects in the database. A change to the Branch ID requires a new version.
- Records the activity associated with an `Activity` step in the database. A change to an `Activity` definition requires a new version.
- Records the trigger activity that occurs in an active workflow in the database. A removal of a trigger requires a new workflow version.
- Records the `messageID` of each workflow message in the database. A modification to a `MessageStep` requires a new workflow version.

If you do modify a workflow, be aware that:

- If you convert a manual step to an automatic step, it can cause issues for an active workflow.
- If you reduce a timeout value, any active workflows that have already hit that step will only wait the previously calculated time.

IMPORTANT: If there is an active workflow on a particular step, do not alter that step without versioning the workflow.

Workflow localization

At the start of the workflow execution, the application evaluates the workflow locale and uses that locale for notes, documents, templates, and similar items. However, it is possible to set a workflow locale that is different from the default application locale through the workflow editor. This change then affects all notes, documents, templates, email messages, and similar items that the various workflow steps create or use.

You can also:

- Set a different locale for any spawned subworkflows.
- Set a locale for a Gosu block that a workflow executes.
- Set Studio to display a workflow step name in a different locale.

See also

- "Set a workflow locale" on page 20
- *Globalization Guide*

Set a workflow locale

About this task

To view or modify the locale for a workflow, do the following:

Procedure

1. Click in the background area of the layout view.
2. In the properties area at the bottom of the screen, in the **Locale** field, enter a valid `ILocale` type to set the overall locale for a workflow.

See also

- *Globalization Guide*

Workflow structural elements

A workflow (or, more technically, a workflow XML script) contains a number of elements that perform a structural function in the workflow. For example, the `<Start>` element designates which workflow step actually initiates the workflow.

The workflow elements include the following:

Element	More information
`<Context>`	"<Context> workflow element" on page 20
`<Finish>`	"<Finish> workflow element" on page 21
`<Start>`	"<Start> workflow element" on page 21

<Context> workflow element

Every workflow begins with a `<Context>` block. You use it to conveniently define symbols that apply to the workflow. You can use these symbols over and over in that workflow. For example, suppose that you extend the `Workflow` entity and add `User` as a foreign key. Then, you can define the symbol `user` for use in the workflow script with the value `Workflow.User`.

Within the workflow, you have access to additional symbols, basically whatever the workflow instance knows about. For example, you can define a symbol such that inserting `claim` into the workflow text actually inserts

`Workflow.Claim`, or inserting `Cancellation` actually inserts `Workflow.PolicyPeriod.Cancellation`, or inserting `dlnqProcess` into the workflow text actually inserts `Workflow.DelinquencyProcess`.

Defining Symbols

You must specify in the context any foreign key or parameter that the workflow subtype definition references. To access the `<Context>` element, select it in the outline view. You add new symbols in the property area at the bottom of the screen.

Field	Description
Name	The name to use in the workflow process for this entity.
Type	The Guidewire entity type.
Value	The instance of the entity being referenced.

<Start> workflow element

The `<Start>` structural block defines the step on which the workflow starts. To set the first step, select `<Start>` in the outline view (center pane). In the properties pane at the bottom of the screen, choose the starting step from the drop-down list of steps. Studio displays the downward point of a green arrow on the step that you chose.

This element can optionally contain Gosu code to set up the workflow or its business data.

<Finish> workflow element

The `<Finish>` structural block is an optional block that contains Gosu code to perform any last cleanup after the workflow reaches an `Outcome Workflow Step`.

Common step elements

It is possible for each step in the workflow to also contain some or all of the following:

- “Enter and exit scripts in workflows” on page 21
- “Asserts in workflows” on page 22
- “Events in workflows” on page 22
- “Notifications in workflows” on page 22
- “Branch IDs in workflows” on page 23

The application **Administration** tab displays the current step for each given workflow instance.

Enter and exit scripts in workflows

A workflow step can have any amount of Gosu code in the `Enter` and `Exit` blocks to define what to do within that step. (Enter scripts are far more common.) To access the enter and exit scripts block, select a workflow step and view the **Properties** tab at the bottom of the screen.

Enter Script	Gosu code that the workflow runs just after it evaluates any `Asserts in Workflows` (conditions) on the step. (That is, if none of the asserts evaluate to `false`. If this happens, the workflow does not run this step.)
Exit Script	Gosu code that the workflow runs as the final action on leaving this step.

For example, you could enter the following Gosu code for the enter script:

```
var msg = "Workflow " + Workflow.DisplayName + "started at " + Workflow.EnteredStep
print(msg)
```

Note: If you rename a property or method, or change a method signature, and a workflow references that property or method in a Gosu field, the application throws a `ParseResultsException`. This is the intended behavior. You must reload the workflow engine to correct the error (**Internal Tools** > **Reload** > **Reload Workflow Engine**).

Asserts in workflows

A step can have any number of `Assert` condition statements. An `Assert` executes just before the `Enter` block. If an `Assert` fails, the workflow throws an exception and handles it like any other kind of runtime exception. To access the **Assert** tab, select a workflow step.

Condition	Each condition must evaluate to a Boolean value.
Error message	If a condition evaluates to false, then the workflow logs the supplied error message.

For example, you could add the following assert condition and error message to log if the assertion fails:

Condition

```
Workflow.currentAction == “start”
```

Error message to log if assertion fails

```
“Some error message if condition is false”
```

Events in workflows

A step can have any number of `Event` elements associated with it. An `Event` runs right after the `Enter` block, and generates an event with the given name and the business object. To access the **Events** tab, select a workflow step.

Entity Name	Entity on which to generate the event. This must be a valid symbol name.	See also: • “<Context> workflow element” on page 20
Event Name	Name of the event to generate. This must be a valid event name.	See also: • *Plugins, Prebuilt Integrations, and SOAP APIs*

For example:

Entity Name	`account`
Event Name	`someEvent`

Notifications in workflows

A step can have any number of non-blocking `Notification` activities. A *notification* in workflow terms is an activity that the application sends out, but which does not block the workflow from continuing. The application only uses it to notify you of something. The workflow generates any notifications immediately after it executes the `Enter` code, if any. See “ActivityStep workflow step” on page 25 for more information on activity generation.

Name	Name of the activity.
Pattern	Activity pattern code. This must be a valid activity pattern as defined through the application.
Init	Optional Gosu code that the workflow executes immediately after it creates the activity. Typically, you use this code to assign the activity. If you do not explicitly assign the activity, the workflow auto-assigns the activity.

For example:

Name	`notification`
Pattern	`general_reminder`

Branch IDs in workflows

A branch is a transition from one step to another. Every branch has an ID, which is its reference name. An ID is necessary because the Workflow instance sometimes needs to persist to the database which branch it is trying to execute. (This can happen, for example, if an error occurs in the branch and the workflow drops into the `Error` state). A branch ID must be unique within a given step.

Generally, as you enter information in a dialog to define a step, you also need to enter branch information as well.

Basic workflow steps

Guidewire uses the following steps (or blocks) to create a workflow:

Workflow step	More information
AutoStep	“AutoStep workflow step” on page 23
MessageStep	“MessageStep workflow step” on page 24
ActivityStep	“ActivityStep workflow step” on page 25
ManualStep	“ManualStep workflow step” on page 26
Outcome	“Outcome workflow step” on page 27

AutoStep workflow step

An `AutoStep` is a step that the application guarantees to finish immediately. That is, it does not wait for anything else such as an activity, a manual trigger, or a timeout before continuing to the next step. The **Workflows** editor indicates an autostep with an arrow icon in the box the represents that step.

Each `AutoStep` step must have at least one “GO” branch. (It can have more than one, but it must have at least one.) Each `GO` branch that leaves an `AutoStep` step—except for the last one listed in the XML code—must contain a condition that evaluates to either Boolean `true` or `false`.

After the `AutoStep` completes its `Assert`, `Enter`, and `Activity` blocks, it goes through its list of `GO` branches (from top to bottom in the XML code):

- It picks the first `GO` branch for which the condition evaluates to `true`.
- If none of the other `GO` branches evaluate to `true`, it picks the last `GO` element (without a condition).

At that point, it executes the `Exit` block and proceeds to the step specified by the chosen `GO` element.

Create an auto workflow step

Procedure

1. Right-click in the workflow workspace, and then click **New AutoStep**.
2. Enter the following fields:

Field	Description
Step ID	ID of the step to create.
ID	ID of a branch leaving this step. It defaults to the **To** value if you do not supply a value.
To	ID of the step to which the workflow goes if the condition specified for this branch evaluates to `true`.

For example:

Step ID	`Step1`
ID	-
To	`DefaultOutcome`

3. Click on your newly created step. It is possible that there are additional tabs to fill out in the properties area at the bottom of the screen. See “Common step elements” on page 21 for information on the various tabs.

MessageStep workflow step

A `MessageStep` is a special-purpose step designed to support messaging-based integrations. It automatically generates and sends a single integration message and then stops the workflow until the message completes. (Typically, this is through receipt of an `ack` return message.) After the message completes, the workflow resumes automatically.

The **Workflows** editor indicates an message step with a mail icon in the box the represents that step.

Just before running the `Enter` block, the workflow creates a new message and assigns it to `Workflow.Message`. Use the `Enter` block to set the payload for the message. After the `Enter` block finishes, the workflow commits its bundle and stops. This commits the message. At this point, the messaging subsystem picks up the message and dispatches it.

If something acknowledges the message (either internal or external), the application stores an optional response string (supplied with the `ack`) on the message in the `Response` field. The application then does the following:

- It copies the message into the `MessageHistory` table
- It updates the workflow to null out the foreign key to the original message and establishes a foreign key to the new `MessageHistory` entity.

It then resumes the workflow (by creating a new work item).

There can be any number of `GO` branches that leave a message step (but only `GO` branches). As with `AutoStep` `Workflow Step`, the workflow evaluates each `GO` condition, and chooses the first one that evaluates to `true`. If none evaluate to `true`, the workflow takes the branch with no condition attached to it.

Create a message workflow step

Procedure

1. Right-click in the workflow workspace, and select **New MessageStep**.
2. Enter the following fields:

Field	Description
Step ID	ID of the step to create.
Destination ID	ID of the destination for the message. This must be a valid message destination ID as defined through the Studio **Messaging** editor.
EventName	Event name on the message.

Field	Description
ID	ID of a branch leaving this step. It defaults to the **To** value if you do not supply a value.
To	ID of the step to which the workflow goes if the condition specified for this branch evaluates to `true`.

For example:

Step ID	`Step4`
Dest ID	`89`
Event Name	`EventName`
ID	
To	`DefaultOutcome`

3. Click on your newly created step. It is possible that there are additional tabs to fill out in the properties area at the bottom of the screen.

See also

- "Common step elements" on page 21

ActivityStep workflow step

An `ActivityStep` is similar an `AutoStep Workflow Step`, except that it can use any of the branch types—including a `TRIGGER` or a `TIMEOUT`—to move to the next step. However, before an `ActivityStep` branches to the next step, it waits for one or more activities to complete. The application indicates the termination of an activity by marking it one of the following:

- Completed (which includes either being approved or rejected)
- Skipped
- Canceled

Activities are a convenient way to send messages and questions asynchronously to users who might not even be logged into the application.

The **Workflows** editor indicates an activity step with a person icon in the box the represents that step.

Within an `ActivityStep`, you specify one or more activities. The workflow creates each defined activity as it enters the step. (This occurs immediately after the workflow executes the **Enter Script** block, if there is one.) The activity is available on all steps.

The only difference between an `Activity` and a `Notification` within a workflow is that:

- An `Activity` pauses the workflow until all the activities in the step terminate.
- A `Notification` does not block the workflow from continuing.

If more than one `Activity` exists on an `ActivityStep`, then the workflow generates all of them immediately after the `Enter` block (along with any events or notifications). The step then waits for all of the activities to terminate. If desired, an `ActivityStep` can also contain `TIMEOUT` and `TRIGGER` branches as well. In that case, if a timeout or a trigger on the step occurs, then the workflow does not wait for all the activities to complete before leaving the step.

After the application marks all the activities as completed, skipped or canceled, the `ActivityStep` uses one or more "GO" branches to proceed to the next step. There can be any number of `GO` branches that leave an activity step. As with `AutoStep Workflow Step`, the workflow evaluates each `GO` condition, and chooses the first one that evaluates to `true`. If none evaluate to `true`, the workflow takes the branch with no condition attached to it.

Notice that it is possible for the condition statement of a `GO` branch to reference a generated `Activity` by its logical name. For instance, it is possible that you want to proceed to a different step depending on whether the application marks the `Activity` as completed or canceled.

Create an activity workflow step

Procedure

1. Right-click in the workflow workspace, and select **New ActivityStep**.
2. The dialog contains the following fields:

Field	Description
Step ID	The ID of the step to create.
Name	Name of the activity.
Pattern	Activity pattern code. This must be a valid activity pattern as defined through the application.
ID	ID of a branch leaving this step. It defaults to the **To** value if you do not supply a value.
To	ID of the step to which the workflow goes if the condition specified for this branch evaluates to `true`.

3. Click on your newly created step and open the **Activities** tab at the bottom of the screen. After you create the `ActivityStep`, you need to create one or more activities. (Each `ActivityStep` must contain at least one defined activity.) These fields on the **Activities** tab have the following meanings:

Name	Name of the activity.
Pattern	Activity pattern `code` value. This must be a valid activity pattern `code` as defined through the application. To view a list of valid activity pattern codes, view the `ActivityPattern` typelist. Only enter a value in the **Pattern** field that appears on this typelist. For example: • `approval` • `approvaldenied` • `general` • `...`

Init	Gosu code that the workflow executes immediately after it creates the activity. Typically, you use this code to assign the activity. If you do not explicitly assign the activity, the workflow auto-assigns the activity. For example, the following initialization Gosu code creates an activity and assigns it `SomeUser` in `SomeGroup`.
	`Workflow.initActivity(Activity)`
	`Activity.autoAssign(SomeGroup, SomeUser)`
	The initialization code creates an activity based on the activity pattern that you set in the **Pattern** field.

ManualStep workflow step

A `ManualStep` is a step that waits for an external `TRIGGER` to be invoked or a `TIMEOUT` to pass. Unlike `AutoStep Workflow Step` or `ActivityStep Workflow Step`, a `ManualStep` must not have, and cannot have, `GO` branches leaving it. However, it can have zero or more `TRIGGER` branches or zero, or more, `TIMEOUT` branches. It must have at least one of these branches. Otherwise, there would be no way to leave this step.

The **Workflows** editor indicates a manual step with an hourglass icon in the box the represents that step.

Manual Step with Timeout

If you specify a *timeout* for this step, then you also need to specify one of the following. (See also "TIMEOUT" on page 31 for more discussion on these two values.)

Time Delta	The amount of time to wait or pause before continuing. Enter an integer number with its units (`3600s`, for example).

Time Absolute	A fixed point in time, as defined by a Gosu expression that resolves to a date. You can use the Gosu code to define the date, as in the following:
	`PolicyPeriod.Cancellation.CancelProcessDate`
	Or, you can use Gosu to calculate the point in time, as in the following:
	`PolicyPeriod.PeriodStart.addDays(-105)`

This defines the terms of the `TIMEOUT` branch that leaves this step. To view these details later, click the branch (the link) between the two steps.

Manual Step with Trigger

If you specify a *trigger* for this step, then you need only enter the branch information. This defines the terms of the `TRIGGER` branch that leaves this step. To view these details later, click the branch (the link) between the two steps.

Create a manual workflow step

Procedure

1. Right-click in the workflow workspace, and select **New ManualStep**.
2. Enter the following fields. What you see in the dialog changes slightly depending on the value you set for **Type** (`TIMEOUT` or `TRIGGER`).

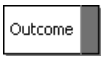
Field	Description
Step ID	ID of the step to create.
Type	Name of the activity.
ID	If you select the following **Type** value: • **Trigger**: A valid trigger key as defined in typelist `WorkflowTriggerKey`. • **Timeout**: ID of a branch leaving this step. It defaults to the **To** value if you do not supply a value.
To	ID of the step to which the workflow goes if the condition specified for this branch evaluates to `true`.
Time Delta	Specifies a fixed amount of time to pause before continuing. For example, the following sets the wait time to 60 minutes (one hour): `3600s`,
Time Absolute	Specifies a fixed point in time. For example, the following sets the point to continue to after the policy `CancelProcessDate`: `PolicyPeriod.Cancellation.CancelProcessDate`

   If the `WorkflowTriggerKey` typelist does not contain any trigger keys, then you do not see the **Trigger** option in the dialog.
3. Click on your newly created step. It is possible that there are additional tabs to fill out in the properties area at the bottom of the screen.

Outcome workflow step

An `Outcome` is a special step that has no branches leading out of it. It is thus a final or terminal step. If a workflow enters any `Outcome` step, it is complete. It is possible (and likely) for a workflow to have multiple outcomes or final steps.

The **Workflows** editor indicates an outcome step with a gray bar in the box to indicate that this is a final step.



After the workflow successfully enters an `Outcome` step (meaning that the workflow successfully executes the `Enter` block of the `Outcome` step), it does the following:

1. The workflow generates all the listed events and notifications.
2. It executes the `<Finish> Workflow Element` block of the workflow process.
3. It changes the state of the workflow instance to `Completed`.

You must structure each workflow script so that its execution eventually and inevitably leads to an `Outcome`. Otherwise, you risk infinitely-running workflows, which means that the load on the application can increase linearly over time, crippling performance.

Create an outcome workflow step

Procedure

1. Right-click in the workflow workspace, and select **New Outcome**.
2. Enter a step ID in the **New Outcome** dialog.
3. Click on your newly created step. It is possible that there are additional tabs to fill out in the properties area at the bottom of the screen.

Step branches

A branch is a transition from one step to another. There are multiple kinds of elements that facilitate branching to another step. They are:

- `GO`
- `TRIGGER`
- `TIMEOUT`

The **Workflows** editor indicates a branch by linking two steps with a line and placing one of the following icons on the line to indicate the branch type.

Type	Icon	Description
`GO`		A branch or transition to another workflow step. It occurs only within an `AutoStep Workflow Step` workflow step. • If there is only a single "GO" branch within the workflow step, branching occurs immediately upon workflow reaching that point. • If there are multiple GO branches within the workflow step, all GO branches (except one) must contain conditional logic. The workflow then determines the appropriate next step based on the defined conditions.
`TRIGGER`		A branch or transition to another workflow element. It occurs only within a `ManualStep Workflow Step` workflow step. Branching occurs only upon manual invocation from outside the workflow.
`TIMEOUT`		A branch or transition to another workflow element. It occurs only within a `ManualStep Workflow Step` workflow step. Branching to another workflow step occurs only after the passing of a specific time interval.

All branch elements contain a **To** value that indicates the step to which this branch leads. It can also contain an optional embedded Gosu block for the workflow to execute if a workflow instance follows that branch.

How a workflow decides which branch to take depends entirely on the type of the branch. However, the order is always the same:

- The workflow executes the `Enter` block for a given step and generates any events, notifications, and activities (waiting for these activities to complete).
- The workflow attempts to find the first branch that is ready to be taken. It starts with the first branch listed for that step in the outline view, then moves to evaluate the next branch if the previous branch is not ready.
- If no branch is ready (which is possible only on a `ManualStep`), the workflow waits for one to become ready.
- After the workflow selects a branch, it runs the `Exit` block, then executes the Gosu block of the branch.
- Finally, the workflow moves to the next step and begins to evaluate it.

Working with branch IDs

Every branch also has an ID, which is its reference name. An ID is necessary because the Workflow instance sometimes needs to persist to the database which branch it is trying to execute. (This can happen if an error occurs in the branch and the workflow drops into the `Error` state). A branch ID must be unique within a given step.

If you do not specify an ID for a branch (which occurs frequently), the workflow uses the value of `nextStep` attribute as a default. This works well except in the special case in which you have more than one branch leading from the same Step A to the same Step B. (This can happen if you want to OR multiple conditions together, or if you want different Gosu in the different branches but the same `nextStep`.) In that case, you must add an ID to each of those branches. Studio complains with a verification error upon loading (or reloading) the workflow scripts if you do not do this.

Do the following to assign an ID to each type of branch:

Type	Action to take
`GO`	Optionally add an ID to a `GO` branch. If you do not provide one, Studio defaults the ID to the value of the `nextStep` attribute. However, Guidewire recommends that you create specific IDs if there are multiple `GO` branches that all move to the same next step.
`TRIGGER`	Always add an ID to a `TRIGGER` branch. Guidewire requires this as you must invoke a trigger explicitly. You must use a value from the `WorkflowTriggerKey` typelist for the branch ID.
`TIMEOUT`	Optionally add an ID to a `TIMEOUT` branch. If you do not provide one, Studio defaults the ID to the value of the `nextStep` attribute.

GO

The simplest kind of branch is `GO`. It appears on `AutoStep`, `ActivityStep` and `MessageStep`. There can be a single `GO` branch or a list of multiple `GO` branches. If there is a single `GO branch,` then you need only specify the **To** field and any optional Gosu code. The workflow takes this `GO` branch immediately as it checks its branches.

The **Workflows** editor indicates a `GO` branch with an arrow icon superimposed on the line that links the two steps. (That is, the initial *From* step and the *To* step to which the workflow goes if the `GO` condition evaluates to `true`.)

To access the dialog that defines the `GO` branch, right-click the starting step—in this case, `CheckOnOrder`—and select **New GO** from the menu. (Studio only displays those choices that are appropriate for that step.) This dialog contains the following fields:

Field	Description
Branch ID	ID of the branch to create
From	ID of the step on the which the GO branch starts.
To	ID of the step on the which the GO branch starts.

It is not necessary to enter a branch ID. However, if you create multiple `GO` branches from a step, then you must enter a unique ID for each branch.

After you create the `GO` branch, click on the link (line) that runs between the two steps. You will see a dialog that contains the following fields:

Field	Description
Branch ID	Automatically generated.
From	Automatically generated. Workflow step ID of the beginning point of the branch.
To	Workflow step ID of the ending point of the branch.
Arrow Visible	Show an arrow head on the branch line to indicate direction.
Description	Description of this branch.
Condition	Must evaluate to either `true` or `false`.
Execution	Gosu code to execute if the workflow takes this branch.

Notice that this branch definition sets a condition. The **From** and **To** fields set the end-points for the branch.

If there are multiple `GO` branches, all the `GO` branches except one must define a condition that evaluates to either Boolean `true` or `false`. The workflow decides which `GO` branch to take by evaluating the `GO` branches from top to bottom (within the XML step definition). It selects the first one whose condition evaluates to `true`. If none of the conditions evaluate to true, then the workflow uses the `GO` branch that does not have a condition. A list of `GO` branches is thus like a `switch` programming block or a series of `if...else...` statements, with the default case at the bottom of the list.

Infinite Loops

Beware of infinite, immediately-executing cycles in your workflow scripts. For example:

From	To
`StepA`	`StepB`
`StepB`	`StepA`

If the steps revolve in an infinite loop, the application catches this only after 500 steps. This can cause other problems to occur.

TRIGGER

Another kind of branch is `TRIGGER`, which can appear in a `ManualStep Workflow Step` or an `ActivityStep Workflow Step`. It also has a **To** field and an optional embedded Gosu block. However, instead of a condition checking to see if a certain Gosu attribute is true, someone or something must manually invoke a `TRIGGER` from outside the workflow infrastructure. (Typically, this happens from either the application interface or from a Gosu call.) Guidewire requires a branch ID field on all `TRIGGER` elements, as outside code uses the ID to manually reference the branch.

Unlike all other the IDs used in workflows, `TRIGGER` IDs are not plain strings but typelist values from the extendable `WorkflowTriggerKey` typelist. This provides necessary type safety, as scripting invokes triggers by ID. However, it also means that you must add new typecodes to the typelist if you create new trigger IDs.

Invoking a trigger

How does one actually invoke a `TRIGGER`? Almost anything can do so, from Gosu rules and classes to the application interface. Typically, in the application, you invoke a trigger though the action of toolbar buttons in a wizard. This is done through a call to the `invokeTrigger` method on `Workflow` instances. (As it is also a scriptable method, you can call it from Gosu rules and the application PCF pages.) See “Synchronicity, transactions, and errors” on page 40 for a discussion of the `invokeTrigger` method and its parameters.

Internally, the method works by updating the (read-only) database field `triggerInvoked` on `Workflow` to save the ID. (See the *Data Dictionary* entry on `Workflow`.)

The application then *wakes up* the workflow instance and the `TRIGGER` inspects the `triggerInvoked` field to see if something invoked the trigger. Depending on how you set the `invokeTrigger` method parameters, the workflow handles the result of the `TRIGGER` either synchronously or asynchronously.

Creating a trigger branch

To access the `TRIGGER` branch dialog, right-click the starting step and select **New Trigger** from the menu. (Studio only displays those choices that are appropriate for that step.) This dialog contains the following fields:

Field	Description
Branch ID	Name of this branch as defined in the **WorkflowTriggerKey** typelist. Select from the drop-down list.
From	Automatically generated. Workflow step ID of the beginning point of the branch.
To	Workflow step ID of the ending point of the branch.

After you create the branch, click on the link (line) that runs between the two steps. You see the following fields, which are identical to those used to define a `GO` branch:

Field	Description
Branch ID	Automatically generated.
From	Automatically generated. Workflow step ID of the beginning point of the branch.
To	Workflow step ID of the ending point of the branch.
Arrow Visible	Show an arrow head on the branch line to indicate direction.
Description	Description of this branch.
Condition	Must evaluate to either `true` or `false`.
Execution	Gosu code to execute if the workflow takes this branch.

Trigger availability

Simply because you define a `TRIGGER` on a `ManualStep` does not mean it is necessarily available. You can restrict trigger availability in the following different ways:

- You can specify user access permission through the use of the `Permission` field.
- You can add any number of `Available` conditions on the **Available** tab to further restrict availability. If the condition expression evaluates to `true`, the trigger is available. Otherwise, it is unavailable.

  For example (from PolicyCenter), the following Gosu code indicates that the workflow can only take this branch if a user has permission to rescind a policy. (The condition evaluates to `true`.)

  ```
  PolicyPeriod.CancellationProcess.canRescind().Okay
  ```

TIMEOUT

Another kind of branch is `TIMEOUT`, which (like `TRIGGER`) can appear on `ManualStep Workflow Step` or an `ActivityStep Workflow Step`. You still have a **To** field and optional Gosu block. However, instead of using a condition to determine how to move forward, the workflow executes the `TIMEOUT` element after the elapse of a specified amount of time.

You can use a `TIMEOUT` in the following ways:

- As the default behavior for a stalled workflow. For example:

  Do x if the application has not invoked a trigger for a certain amount of time.

- As a deliberate delay. For example:
  Go to sleep for 35 days.

You can specify the time to wait using one of the following attributes. (Studio complains if you use neither or both.)

- `timeDelta`
- `timeAbsolute`

The time delta value

The **Time Delta** value specifies an amount of time to wait, starting from the time the Workflow instance successfully enters the step. (The wait time starts immediately after the workflow executes the **Enter Script** block for the step.) You specific the time to wait with a number and a unit, for example:

- `100s` for 100 seconds
- `15m` for 15 minutes
- `35d` for 35 days

You can also combine numbers and units, for example, `2d12h30m` for 2 days, 12 hours, and 30 minutes.

The time absolute value

Often, you do not want to wait a certain amount of time. Instead, you want the step to time out after passing a certain point relative to a date in the business model (for example, five days after a specific event occurs). In that case you can set the **Time Absolute** value, which is a Gosu expression that must resolve to a date.

IMPORTANT: Do not use the current time in a **Time Absolute** expression. The workflow reevaluates this expression each time it checks `TIMEOUT`. For example, the time-out never ends for the following expression, `java.util.Date.CurrentDate + 1`, as the expression always evaluates to the future.

Creating a timeout branch

You use the **Workflows** editor to define a `Timeout` branch. To access the `Timeout` branch dialog, right-click the starting step and select **New Timeout** from the menu. You must enter either a time absolute expression or a time delta value. This dialog contains the following fields:

Field	Description
Branch ID	Name you choose for this branch.
From	Automatically generated. Workflow step ID of the beginning point of the branch.
To	Workflow step ID of the ending point of the branch.
Time Delta	Time to wait, starting from the time the Workflow instance successfully enters the step.
Time Absolute	Gosu expression that must resolve to a fixed date.

After you create the branch, click on the link that runs between the two steps. You see the following fields:

Field	Description
Branch ID	Automatically generated.
From	Automatically generated. Workflow step ID of the beginning point of the branch.
To	Workflow step ID of the ending point of the branch.
Arrow Visible	Show an arrow head on the branch line to indicate direction.
Time Delta	Time to wait, starting from the time the Workflow instance successfully enters the step.

Field	Description
Time Absolute	Gosu expression that must resolve to a fixed date.
Execution	Gosu code to execute if the workflow takes this branch.

Creating new workflows

To create a new workflow, you can do the following:

Action	Description
Clone an Existing Workflow	Creates an exact copy of an existing workflow type, with the same name but with an incremented version number. (This process clones the workflow with highest version number, if there multiple versions already exist.) Perform this procedure if you merely want a new version of an existing workflow.
Extend an Existing Workflow	Creates a new (blank) workflow with a name of your choice based on the workflow type of your choice.

Clone an existing workflow

About this task

Cloning an existing workflow is a relatively simple process. Also, if you clone an existing, fully built workflow, then you can leverage the work of the original workflow. However, you can only clone existing workflow types. You cannot use this method to create a new workflow type.

Procedure

1. Open the **Workflows** node in the **Project** window tree.
2. Select an existing workflow type, right-click and select **New** > **Workflow** from the menu.

Results

Studio creates a cloned, editable copy of the workflow process and inserts it under the workflow node with an incremented version number. You can then modify this version of the workflow process to meet your business needs.

Extend an existing workflow

About this task

To extend an existing workflow, you must create an `eti` (extension) file and populate it correctly. To assist you, Studio provides a dialog in which you can enter the basic workflow information. You must then enter this information in the `eti` file.

Procedure

1. First, determine the workflow type that you want to extend.
2. Select **Workflows** in the **Project** window, right-click and select **Create metadata for a new workflow subtype** from the menu.
3. In the **New Workflow subtype metadata** dialog, enter the following:

Field	Description
Entity	The workflow object to create.
Supertype	The type or workflow to extend. You can always extend the `Workflow` type, from which all subtypes extend.

Field	Description
Description	Optional description of the workflow.
Foreign keys	Click the **Add** button to enter any foreign keys that apply to this workflow object.

4. Click **Gen to clipboard**. This action generates the workflow metadata information in the correct format and stores on the clipboard.
5. Expand the **Extensions** folder in the **Project** window.
6. Right-click the **Entity** folder and select **New** > **Entity** from the menu.
7. Enter the name of the file to create in the **New File** dialog. Enter the same value that you entered in the **New Workflow subtype metadata** dialog for **Entity** and add the `eti` extension. Studio then creates a new `<entity>.eti` file.

   Open this file, right-click, and choose **Paste** from the menu. Studio pastes in the metadata workflow that you created in a previous step.

   For example, if you extend `Workflow` and create a new workflow named `NewWorkflow`, then you must create a new `NewWorkflow.eti` file that contains the following:

   ```
   <?xml version="1.0"?>
     <subtype desc="" entity="NewWorkflow" supertype="Workflow"/>
   ```

8. (Optional) To provide the ability to localize the new workflow, add the following line of code to this file (as part of the `subtype` element):

   ```
   <typekey desc="Language" name="Language" typelist="LanguageType"/>
   ```

   Continuing the previous example, you now see the following:

   ```
   <?xml version="1.0"?>
     <subtype desc="" entity="NewWorkflow" supertype="Workflow">
     <typekey desc="Language" name="Language" typelist="LanguageType"/>
   <subtype>
   ```

9. Stop and restart Guidewire Studio so that it picks up your changes.
   - You now see `NewWorkflow` listed in the **Workflow** typelist.
   - You now see an `NewWorkflow` node under **Resources** > **Workflows**.
10. Select the `NewWorkflow` node under **Workflows**, right-click and select **New Workflow Process** from the menu. Studio opens an empty workflow process that you can modify to meet your business needs.

Extending a workflow: a simple example

This simple examples illustrates the following steps:

- “Step 1: Extend an existing workflow object” on page 34
- “Step 2: Create a new workflow process” on page 35
- “Step 3: Populate Your workflow with steps and branches” on page 35

Step 1: Extend an existing workflow object

To extend an existing workflow object, review the steps outlined in “Extend an existing workflow” on page 33. For this example, you create a new `ExampleWorkflow` object by extending (subtyping) the base `Workflow` entity.

To extend a workflow object

1. Create a new `ExampleWorkflow.eti` file and enter the following:

   ```
   <?xml version="1.0"?>
     <subtype desc="" entity="ExampleWorkflow" supertype="Workflow">
   ```

```
  <typekey desc="Language" name="Language" typelist="LanguageType"/>
</subtype>
```

2. Close and restart Studio.

You now see an `ExampleWorkflow` entry added to the `Workflow` typelist and a new `ExampleWorkflow` workflow type added to **Workflows** in the **Resources** tree.

After completing this task, complete "Step 2: Create a new workflow process" on page 35.

Step 2: Create a new workflow process

Before beginning this task, complete "Step 1: Extend an existing workflow object" on page 34.

Next, you need to create a new workflow process from your new `ExampleWorkflow` type.

To create a new workflow process

1. Select `ExampleWorkflow` from **Workflows** in the **Project** window.
2. Right-click and select **New Workflow** from the menu.

Studio opens an outline view and layout view for the new workflow process:

- The outline view contains the few required workflow elements.
- The layout view contains a default outcome (`DefaultOutcome`).

After completing this task, complete "Step 3: Populate Your workflow with steps and branches" on page 35.

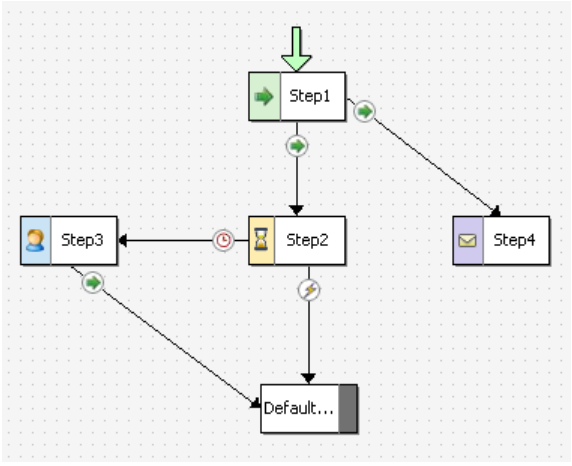
Step 3: Populate Your workflow with steps and branches

Before beginning this task, complete "Step 2: Create a new workflow process" on page 35.

Finally, to be useful, you need to add outcomes, steps, and branches to your workflow. This examples creates the following:

- A `Step1` (`AutoStep Workflow Step`) with a default `GO` branch to the `DefaultOutcome` step, which you designate as the first step in the `<Start> Workflow Element` element
- A `Step2` (`ManualStep Workflow Step`) with a `TRIGGER` branch to the `DefaultOutcome` step
- A `Step3` (`ActivityStep Workflow Step`) with a `GO` branch to the `DefaultOutcome` step
- A `TIMEOUT` branch from `Step2` to `Step3`, with a `5d` time delta set
- A `Step4` (`MessageStep Workflow Step`) with a "GO" branch from `Step1` to `Step4`

The example workflow looks similar to the following:



This example does not actually perform any function. It simply illustrates how to work with the dialogs of the **Workflows** editor.

To add steps and branches to a workflow

1. Right-click within an empty area in the layout view and select **New AutoStep** from the menu:
   - For **Step ID**, enter `Step1`.
   - Do not enter anything for the other fields.

Studio adds your autostep to the layout view and connects `Step1` to `DefaultOutcome` with a default "GO" branch.

1. Select **<Start> Workflow Element** in the outline view (middle pane):
   - Open the **First Step** drop-down in the property area at the bottom of the screen.
   - Select `Step1` from the list. This sets the initial workflow step to `Step1`.
   - Save your work.
2. Right-click within an empty area in the layout view and select **New ManualStep** from the menu:
   - For **Step ID**, enter `Step2`.
   - For branch **Type**, select `TRIGGER`.
   - For trigger **ID**, select `Cancel`.

The **ID** value sets a valid trigger key as defined in typelist `WorkflowTriggerKey`. If `Cancel` does not exist, then choose another trigger key. If no trigger keys exist in `WorkflowTriggerKey`, then you must create one before you can select `TRIGGER` as the type.

1. Select the `GO` branch (the line) leaving `Step1`:
   - In the property area at the bottom of the screen, change the **To** field from `DefaultOutcome` to `Step2`. Studio moves the branch to link the specified steps.
   - Realign the steps for more symmetry, if you choose.
2. Right-click within an empty area in the layout view and select **New ActivityStep** from the menu:
   - For **Step ID**, enter `Step3`.
   - For **Name**, enter `ActivityPatternName`.
   - For **Pattern**, enter `NewActivityPattern`.
3. Select `Step3`, right-click, and select **New TIMEOUT** from the menu:
   - For **Branch ID**, enter `TimeoutBranch`.
   - For **Time Delta**, enter `5d`. This sets the absolute time to wait to five days.
   - For **To**, select `Step3`.

Studio adds a branch from `Step2` to `Step3` and adds the timeout symbol to it.

1. Right-click within an empty area in the layout view and select **New MessageStep** from the menu:
   - For **Step ID**, enter `Step4`.
   - For **Dest ID**, enter `89` (or any valid message destination ID).
   - For **Event Name**, enter `EventName`.

Studio adds the step to the layout view and creates a link between `Step4` and `DefaultOutcome`.

1. Select the new link from `Step4` to `DefaultOutcome`.
   - In the property area at the bottom of the screen, change **Arrow Visible** to `false` to delete this link.

Studio removes the link (branch).

1. Select `Step1`, right-click, and select **New GO** from the menu:
   - For **Branch ID**, enter `Step4`.
   - For **To**, select `Step4`.

Studio adds the new `GO` branch between `Step1` and `Step4`.

Instantiating a workflow

It is not sufficient to create a workflow. Generally, you want to do something moderately useful with it. To perform work, you must instantiate your workflow and call it somehow.

Suppose that you create a new workflow and call it, for lack of a better name, `HelloWorld1`. You can then instantiate your workflow using the following Gosu:

```
var workflow = new HelloWorld1()
workflow.start()
```

Starting a workflow

There are multiple workflow `start` methods. The following list describes them.

`start()`	Starts the workflow.
`start(version)`	Starts the workflow with the specified process version.
`startAsynchronously()`	Starts the workflow asynchronously.
`startAsynchronously(version)`	Starts the workflow with the specified process version asynchronously.

See also

- "Workflow versioning" on page 18

Logging workflow actions

There are several different Gosu statements that you can use to view workflow-related information.

`gw.api.util.Logger.logInfo`	Statement written to the application server log
`Workflow.log`	Statements viewable in the application **Workflow** console

See Also

- "Workflow debugging, logging, and testing" on page 45

A simple example of instantiation

The following example creates a trivial workflow named `HelloWorld1`. The objective of this example is not to show the branching structure that you can create in workflow. Rather, the purpose of this exercise is to construct the workflow, trigger the workflow, and examine the workflow in the application **Workflow** console. The example keeps the workflow as simple as possible. The workflow consists of the following components:

- `<Context> Workflow Element`
- `<Start> Workflow Element`
- `Step1`
- `Step2`
- `DefaultOutcome`
- `<Finish> Workflow Element`

A Simple ClaimCenter Example

Note: This example uses business entities and rules that apply specifically to the Guidewire ClaimCenter application. However, the particular business objects are not important. What is more important is how you create and instantiate a workflow process.

For the workflow to run and do some work and appear on the workflow console, the example instantiates it from a Claim Update rule. If you attempt to instantiate the workflow from a link or button on a Claim view screen (**Claim Summary**, for example) the workflow executes but does not update anything. Also, it does not appear in the **Workflow** console.

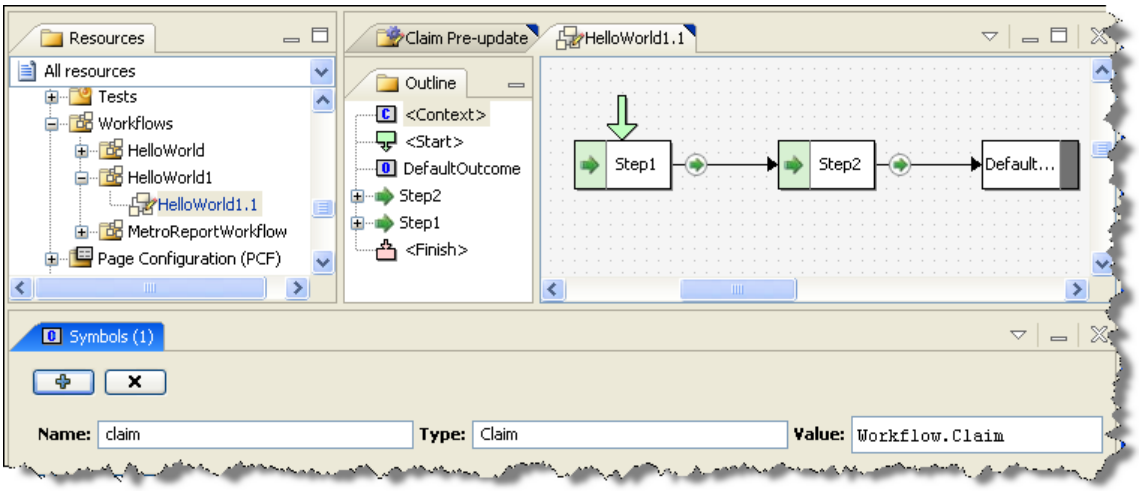
To cause updates to happen, the example instantiates the workflow from an **Edit** screen in ClaimCenter. It then calls a Claim Preupdate Rule in Studio.

To create a simple workflow and instantiate it

1. Create a `HelloWorld1.eti` file (in **Extensions** > **Entity**) and populate it with the following:

```
<?xml version="1.0"?>
<subtype desc="HelloWorld 1 Example Workflow"
         entity="HelloWorld1"
         supertype="ClaimWorkflow">
  <typekey desc="Language" name="Language" typelist="LanguageType"/>
</subtype>
```

2. Stop and restart Studio.
3. Select your new workflow type from the **Workflows** node. Right-click and select **New** > **Workflow Process**.
4. Create a simple workflow process similar to the following. It does not need to be complex, as it simply illustrates how to start a workflow from the ClaimCenter interface.



Notice that it has a `claim` symbol set in "<Context> Workflow Element".

1. For `Step1`, add the following to the `Enter` block for that step:

```
gw.api.util.Logger.logInfo( "HelloWorld1 step 1, step called ClaimNumber " + claim.ClaimNumber)
Workflow.log( "HelloWorld Step 1", "HelloWorld1 step 1 entered: Claim Number " + claim.ClaimNumber )
```

2. For `Step2`, add the following to the `Enter` block for that step:

```
gw.api.util.Logger.logInfo( "HelloWorld1 step 2, step called ClaimNumber " + claim.ClaimNumber)
Workflow.log( "HelloWorld Step 2", "HelloWorld1 step 2 entered: Claim Number " + claim.ClaimNumber )
```

3. Create a simple Claim Pre-Update rule similar to the following:
   - The *rule condition* specifies that the application instantiates the workflow only if the claim `PermissionRequired` property is set to `fraudriskclaim`.

- The *rule action* instantiates the `HelloWorld1` workflow. It first tests for an existing `HelloWorld1` workflow that is not in the completed state and that has the same claim number as the one being updated. If it does not find a matching workflow, then the application instantiates `HelloWorld1` and logs the information.

Rule Conditions:

```
claim.PermissionRequired=="fraudriskclaim"
```

Rule Actions:

```
gw.api.util.Logger.logInfo( "Entering Pre-Update" )

 var hw_wf = claim.Workflows.firstWhere( \ c -> c.Subtype == "HelloWorld1"
        && (c as entity.HelloWorld1).State !="completed"
        && (c as entity.HelloWorld1).Claim.ClaimNumber==claim.ClaimNumber)

if (hw_wf == null) {
  gw.api.util.Logger.logInfo( "## Studio instantiating HelloWorld1 and starting it!" )
  var workflow = new entity.HelloWorld1()
  workflow.Claim = claim
  workflow.start()
}
```

1. Log into ClaimCenter and open any sample claim.
2. Navigate to the **Claim Summary** page, then select the **Claim Status** tab.
3. Click **Edit** and set the **Special Claim Permission** value to `Fraud risk`.
4. Click **Update**. This action triggers the `HelloWorld1` workflow.

To view the server console

1. Navigate to the application server console.
2. View the logger statements.

To view the Workflow console

1. Log into ClaimCenter using an administrative account.
2. Navigate to the **Administration** tab and select **Workflows** from the left-side menu.
3. Click **Search** in the **Find Workflows** screen. You do not need to enter any search information. Studio displays a list of workflows, including `HelloWorld1`.
4. Select `HelloWorld1` from the list and view its details.

The workflow engine

The workflow engine is responsible for processing a workflow. It does this by looking up and executing the appropriate workflow process script. This script (often just called workflow process or workflow script) is an XML file that the Studio workflow editor generates, and which you manage in Studio. The base configuration workflow scripts are in the `modules/configuration/config/workflow` directory.

Distributed execution

The application uses a work queue to handle workflow execution. This, in simple terms, means that you can have a whole cluster of machines that:

- Wake up internal `Workflow` instances,
- Advance them as far as they can go,
- Then, let them go back to sleep if they need to wait on a timeout or activity.

Asynchronous workflow execution always works the same way:

1. The application creates a `WorkflowWorkItem` instance to advance the workflow.
2. The worker instance picks up the work item.
3. The work item retrieves the workflow and advances it as far as possible (to a `ManualStep` or `Outcome`).

You can create a work item in any of the following different ways:

- By a call to the `AbstractWorkflow.startAsynchronously` method
- By invoking a trigger with `asynchronous = true`
- By completing a workflow-linked activity
- By the `Workflow` batch process, which queries for active workflows waiting on an expired timeout
- By a call to `AbstractWorkflow.resume`, typically initiated by an administrator using the workflow management tool

After the workflow advances as far as it can, the application deletes the work item and execution stops until there is another work item.

Synchronicity, transactions, and errors

Error handling for workflows depends on whether the workflow is running synchronously or asynchronously.

Synchronous and asynchronous workflow

It is possible to start workflow either synchronously or asynchronously. To do so, use one of the `start` methods described in “Instantiating a workflow” on page 37. These methods are:

- `start()`
- `start(`*`version`*`)`
- `startAsynchronously()`
- `startAsynchronously(`*`version`*`)`

If a workflow runs synchronously, then it continues to go through one `AutoStep Workflow Step` or `ManualStep Workflow Step` after another until it arrives at a stop condition. This advance through the workflow can encompass one or multiple steps. The workflow executes the current step unless there is an error, and then continues to the next step, if possible. There can be many different reasons that a workflow cannot continue to the next step.

For example, the workflow can enounter:

- An activity step (`ActivityStep Workflow Step`). This can result in the creation of one or more activities, causing the workflow to pause until the closure of all the activities.
- A communication step (`MessageStep Workflow Step`), which can result in a message being sent to another system, causing the workflow to wait until receiving a response.
- A step that stipulates a timeout (`ManualStep Workflow Step`), which causes the workflow to wait for the timeout to complete.
- It can encounter a step that requires a trigger (`ManualStep Workflow Step`), which causes the workflow to wait until someone (or something) activates the trigger.

Ultimately, the workflow can run until it reaches an `Outcome Workflow Step`, at which point, it is done.

After pausing, the workflow waits for one of the following to occur:

- If waiting on one or more activities to complete, it continues after the closure of the last activity.
- If waiting for an acknowledgment of a message, it continues after receiving the appropriate response.
- If waiting on a timeout, it continues after the timeout elapses.
- If waiting on an external trigger, then someone or something must manually invoke a `TRIGGER` from outside the workflow infrastructure. This can happen either from the application interface (a user clicking a button) or from Gosu. In either case, this is done through a call to the `invokeTrigger` method on a `Workflow` instance.

The action of completing an activity or the receipt of a message response automatically creates a work item to advance the workflow. A background batch process checks for timeout elements. It is responsible for finding timed-out workflows that are ready to advance and creating a work item to advance them.

The invokeTrigger method

If a user (or Gosu code) invokes an available trigger (`TRIGGER`) on a `ManualStep Workflow Step`, the workflow can execute either synchronously or asynchronously. A Boolean parameter in the `invokeTrigger` method determines the execution type. This method takes the following signature:

```
void invokeTrigger(WorkflowTrigger triggerKey, boolean synchronous)
```

For example (from PolicyCenter):

```
policyPeriod.ActiveWorkflow.invokeTrigger( trigger, false )
```

The `trigger` parameter defines the `TRIGGER` to use. This must be a valid trigger defined in the `WorkflowTriggerKey` typelist.

The `synchronous` value in this method has the following meanings:

`true`	(Default) Instructs the workflow to immediately execute in the current transaction and to block the calling code until the workflow encounters a new stopping point.
`false`	Instructs the workflow to run in the background, with the calling code continuing to execute. The workflow continues until it encounters a new stopping point.

Trigger availability

For a trigger to be available, the workflow execution sequence must select a branch for which both of the following conditions are true:

- A trigger must exist on the step.
- There is no other determinable path (which usually means that no timeout has already expired).

Thus, if both of these conditions are true, after an invocation to the `invokeTrigger` method, the Workflow engine starts to advance the workflow from the selected branch again.

Invoking a trigger

Invoking a trigger (either synchronously or asynchronously) does the following:

1. It updates the workflow. Any changes made to a transaction bundle that were committed by the actual invocation of the trigger, are committed.
2. It causes the workflow to create a log entry of the trigger request. If there is an error in the workflow advance, any request to the workflow to resume causes the process to start again.
3. If the Workflow engine determines that all the preconditions are met for continuing, it does the following:
   a. It determines the *locale* in which to execute.

      This is the locale that the application uses for display keys, dates, numbers, and other similar items. By default, this is the application default locale. It is important for the Workflow engine to determine the locale as it is possible to override this locale for any specific workflow subtype. You can also override the locale in the workflow definition on the workflow element.
4. It steps through each of the workflow steps (meaning that it performs all the actions within that step) until it cannot keep going.
5. It commits the transaction associated with the executed steps to the database.

Error handling and transaction rollback

If there is an error during a workflow step, the Workflow engine rolls the database back and leaves it in the state that it was previouosly in. If working with an external system, you need to do one of the following:

- Design the services in the external system.
- Use the Guidewire message subsystem to keep an external system state in synchronization with the application database state.

It is important to understand whether a workflow executes synchronously or asynchronously because the type of execution affects errors and transaction rollbacks.

Execution type	Application behavior
Synchronous	If any exception occurs during *synchronous* execution, even after the workflow has gone through several steps, the application rolls back all workflow steps, along with everything else in the bundle. The error cascades all the way up to the calling code, which is the code that started the workflow or invoked the trigger on the workflow. • If you start the workflow or invoke the trigger from the the application interface, the application displays the exception in the interface. • If some other code started the workflow, that code receives the exception.
Asynchronous	If any exception occurs during *asynchronous* execution (as it executes in the background), the application logs the exception and rolls back the bundle in a similar manner to the synchronous case. The application then handles workflow retries in the standard way through the worker. The application leaves the work item used to advance the workflow checked out. It simply waits until the `progressinterval` defined for the workflow work queue expires. At that point, a worker picks it up and retries it. The work queue configuration limits the number of retries. If all retries fail, the application marks the work item as failed and it puts the workflow into the `Error` state. A workflow in the Error `state` merely sits idle until you restore it from the **Administration** tab within the application. Restoring the workflow creates another work item. After you manually restore a workflow from an `Error` to an `Active` state, it again tries to resume whatever it was doing as it left off, such as: • Entering the step • Following the branch • Attempting to perform whatever it was doing at the time the exception occurred Of course, if you have not corrected the problem that caused the error, then the workflow can drop back into `Error` state again. It does so only after the work item performs its specified number of retries.

Workflow guidelines

In practice, Guidewire recommends that you keep the following guidelines in mind as you work with workflows:

- If you invoke a workflow `TRIGGER`, do so synchronously if you need to make immediate use (in code) of the results of that trigger. For this reason, the application rendering framework typically always invokes the trigger synchronously. But notice that you only get immediate results from an `AutoStep Workflow Step` that might have executed. If the workflow encounters a `ManualStep Workflow Step` or an `ActivityStep Workflow Step`, it immediately goes into the background.
- If you complete an activity, it does not synchronously (meaning immediately) advance the workflow. Instead, a background process checks for workflows whose activities are complete and which are therefore ready to move forward. Guidewire provides this behavior, as otherwise, if an error occurs, the user who completes the activity sees the error, which is possibly confusing for that user.
- If you invoke a workflow `TRIGGER` from code that does not necessarily care whether there was a failure in the workflow, you need to invoke the `TRIGGER` asynchronously. (You do this by setting the `synchronous` value in the workflow method to `false`.) That way, the workflow advances in the background and any errors it encounters force

the workflow into the `Error` state. The exception does not affect the caller code. However, the calling code creates an exception if it tries to invoke an unavailable or non-existent workflow `TRIGGER`. Messaging plugins, in particular, need to always invoke triggers asynchronously.

Workflow subflows

A workflow can create another child workflow in Gosu by using the scriptable `createSubFlow` method on `Workflow`. There are multiple versions of this method:

```
Workflow.createSubFlow(workflow)
Workflow.createSubFlow(workflow, version)
```

A subflow has the same foreign keys to business data as the parent flow. It also has an edge foreign key reference to the caller `Workflow` instance, accessed as `Workflow.caller`. (If internal code, and not some other workflow, calls a *macro* workflow, this field is `null`.)

Each workflow also has a `subFlow` s array that lists all the flows created by the workflow, including the completed ones. (This array is empty for workflows that have yet to create any subflows.) The Gosu to access this array is:

```
Workflow.SubFlows
```

You can use subflows to implement simple parallelism in internal workflows, which is otherwise impossible as a single workflow instance cannot be in two steps simultaneously. For example, it is possible for the macro flow to create a subflow in step A. It can then leave this subflow to do its own work, and only wait for it to complete in step E. It is your responsibility as the one configuring the macro workflow to decide how to react if a subflow drops into `Error` mode or becomes canceled for some reason.

See also

- *Globalization Guide*

Workflow administration

You can administer workflow in any of the following ways:

- Through the application **Administration** > **Workflows** page
- Through the command line, such as running a batch process to purge the workflow logs
- Through class `gw.webservice.workflow.IWorkflowAPI`, which the command line uses

The most likely need for using the application **Administration** interface is error handling. Errors can include the following:

- A few workflows fail
- In a worst case scenario, thousands of workflows fail simultaneously

Finding workflows that have not failed but have been idling for an extremely long time is also likely. A secondary use is just looking at all the current running flows to see how they work. Guidewire therefore organizes the **Administration** interface for workflow around a search screen for searching for workflow instances. You can filter the search screen by instance type, state (especially `Error` state), work item, last modified time, and similar criteria.

A user with administrative permissions can search for workflows from the **Administration** > **Workflows** page. However, to actually manage workflows, that user must have the `workflowmanage` permission.

With the correct permission, you can do the following from the **Administration** > **Workflows** page:

- Search for a specific workflow or see a list of all workflows:
- Look at an individual workflow details, for example:
  - View its log and current step and action

  - View any open activities on the workflow
- Actively manage a workflow

Manage workflow

If you have the `workflowmanage` permission, the application enables the following choices on the **Find Workflows** page:

- **Manage selected workflows** (active after you select one or more workflows)
- **Manage all workflows** (active at all times with the correct permission)

Choosing one of these options opens the **Manage Workflows** page. This page presents a choice of workflow and step appropriate commands that you can execute. It is only possible to select one command (radio button) at a time. Choosing either **Invoke Trigger** or **Timeout Branch** provides further selection choices.

Command	Description
Wait - max time (secs)	Select and enter a time to force the workflow to wait until either that amount of time has expired or the currently active work item is no longer active. (The work item has failed or has succeeded and has been deleted.) This option is only available if there is a currently available work item on this workflow.
Invoke Trigger	Select to chose a workflow trigger to invoke. After selecting this command, the application presents a list of available triggers from which to choose, if any are available on this workflow.
Suspend	Select to suspend any active workflows that are currently selected in the previous screen. After you execute this command, the application suspends the selected workflows. This action is appropriate for all workflow and steps. However, the application executes this command only against active workflows.
Resume	Select to resume workflow execution of any suspended workflows that are currently selected in the previous screen. This action is appropriate for all workflows and steps.
Timeout branch	Select to choose a workflow timeout branch. After selecting this command, the application presents a list of timeout branches from which to choose, if any are available on this workflow.

After you make your selection and add any relevant parameters, clicking **Execute** immediately executes that command. Using these commands, you can:

- Restore workflows from the `Error` or `Suspended` state back to the `Active` state. However, if you have not corrected the underlying error, presumably a scripting error, the workflow might drop right back into `Error` mode.
- Force a waiting workflow to execute:
  - By setting the specific timeout branch
  - By setting a specific trigger
- Force an active workflow to wait for a specified amount of time

Workflow Statistics tab

The application collects workflow statistics periodically and captures the elapse and execution time for individual workflow types and steps. You can search by workflow type and date range.

Workflow and server tools

Those with access to the Server Tools, can also access the following:

Batch Process Info	Use to view information on the last run-time of a writer, and to see the schedule for its next run-time. From this page, you also have the ability to stop and start the scheduling of the writer.
Work Queue Info	Use to view information on a writer, what items it picked up and the workers. From this page, you also have the ability to notify, start and stop workers across the cluster.

Workflow debugging, logging, and testing

Debugging a workflow is a more challenging task than debugging the standard application interface flow, as most of the work happens asynchronously, away from any user. Currently, there is no way to set breakpoints in a workflow in a similar fashion to how you can set a breakpoint for a Gosu rule or class.

However, InsuranceSuite does provide workflow logging. Each instance of a workflow has its own internal log that you can view from within the application. (You access this log from **Workflows** page by first by finding a workflow, then by clicking on the **Workflow Type** link.) This log includes successful transitions in the current step and action. It also contains any exceptions. Workflow can access this log, but the application commits these log message only with the bundle.

Use the following logging method, for example, in an **Enter Script** block to log the current workflow step:

```
Workflow.log(summary, description)
```

The method returns the log entry (`WorkflowLogEntry`) that you can use for additional processing:

```
var workflowLog = Workflow.log("short description", "stack trace ...")
var summary = workflowLog.summary
```

Process logging

The following logging categories can be useful:

Category	Use for
`WorkQueue`	A category for general logging from the work queue.
`WorkQueue.Instrumented`	Capturing of runner state for a specific execution of the runner.
`WorkQueue.Item`	Logging (by workers) of each work item executed at the “info” level.
`WorkQueue.Runner`	Logging runners.

To write every message logged by every workflow, set the logging level of the workflow logger category to DEBUG (in file `log4j2-json.xml`). The directive in the `log4j2-json.xml` file is:

```
log4j.category.Server.workflow=DEBUG
```

Workflow testing in PolicyCenter and BillingCenter

It can often be difficult to test a workflow. This is especially true for one that is asynchronous and that requires the workflow to wait a specific amount of time before advancing to the next step. To facilitate testing, the application supports a testing clock that permits the advancing of time (for development-mode servers only). If you have permission, you can access this functionality from the (unsupported) the application **Internal Tools** page. Depending on which clock you define in the `ITestingClock.xml` plugin registration file, you can do one of the following:

- Increment the current clock by a given period.
- Change the setting of the clock to a specific time. The clock remains at that time until another specific time is set.

Enable the ITestingClock plugin in PolicyCenter and BillingCenter

Before you begin

If the `ITestingClock` plugin is not already implemented, then you need to implement it.

Procedure

1. Start Guidewire Studio.
2. In the **Project** window, navigate to **configuration** > **config** > **Plugins** > **registry**.
3. If **ITestingClock** does not exist in the registry, create the plugin by performing the following steps:
   a) Right-click **registry** and click **New** > **New Plugin**.
   b) In the **Name** field, type `ITestingClock`.
   c) In the **Interface** field, type `ITestingClock`.
   d) Click **OK**.
4. If necessary, set the plugin class to the internal offset testing clock by performing the following steps:
   a) In the Plugins Registry editor, click the "+" icon below the **Interface** field. Select **Add PluginAdd Java Plugin**.
   b) In the **Java Class** field, type the following class name.

      ```
      com.guidewire.pl.plugin.system.internal.OffsetTestingClock
      ```

      Note: The OffsetTestingClock class is located in the pl-50.9.1.jar file.
5. In the **Project** window, navigate to **configuration** > **config**, and then open `config.xml` for editing:.
   a) Set parameter `EnableInternalDebugTools` to `true`.

      You must enable the internal debug tools in order to use the **Testing System Clock** screen.
   b) Set parameter `ClusterClockChangeDelaySeconds` to an integer value between 0 and 30.

      This parameter controls how long the cluster waits to ensure all members have received the clock update request and are prepared for the time change. The default is 30.
6. Save your changes.
7. Stop, and then restart the application.